

Technische Universität Berlin
Fakultät IV
Fachgebiet Agententechnologien in betrieblichen Anwendungen und der Telekommunikation
(AOT)

Projektbericht

Entwicklung einer schnellen KI für das Spiel
Wächter und Türme
Gruppe AD

Carsten Frédéric Schlieben (0485860)
Jonas Moritz Müller (0481804)
Elia Ruben Kraemer (0489849)

Git-Repository

20. Mai 2026

Abstract

In diesem Projektbericht wird die Entwicklung einer schnellen und spielstarken Künstlichen Intelligenz für das Spiel *Wächter und Türme* dokumentiert. Ziel war es, unterschiedliche KI-Strategien zu entwerfen, umzusetzen und hinsichtlich ihrer Spielstärke und Effizienz zu evaluieren. Dabei wurden sowohl einfache Heuristiken als auch fortgeschrittene Optimierungsverfahren berücksichtigt. Der Bericht erläutert die Spielregeln, die verwendete Programmiersprache, das modulare Systemdesign sowie die Methodik zur Entscheidungsfindung und Bewertung. Abschließend werden die Ergebnisse aus Tests und Turnieren reflektiert und ein Ausblick auf mögliche Weiterentwicklungen gegeben.

Inhaltsverzeichnis

Eidesstattliche Erklärung	1
1 Einführung	2
1.1 Motivation und Ziele	2
1.2 Aufbau des Berichts	2
2 Grundlagen	3
2.1 Spielregeln	3
2.2 Verwendete Programmiersprache	4
2.3 Projektmanagement	4
3 Ergebnisse	5
3.1 Dummy KI	5
3.1.1 Bitboards	6
3.1.2 Zuggenerator	7
3.1.3 Tests und Benchmarks	7
3.2 Basis KI	7
3.2.1 ein eigener Client	8
3.2.2 Performance-Optimierung durch Parallelisierung	8
3.2.3 Refactoring	9
3.3 Erweiterte KI	11
3.3.1 Zugsortierung	12
3.3.2 Aspiration Window	12
3.3.3 Transposition Tables - TT	12
3.3.4 Principal Variation Search - PVS	14
3.4 Optimierte KI	14
3.4.1 Auswahl der Komponenten	14
3.4.2 Evaluation	16
3.4.3 Tuning der Evaluationsfunktion	17
3.4.4 Teilnahme am Wettbewerb	18
4 Diskussion und Ausblick	19
5 Literatur	20

Eidesstattliche Erklärung

Hiermit versichern wir, dass wir diese Projektarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Die Stellen unserer Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen sind, haben wir in jedem Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht. Dasselbe gilt sinngemäß für Tabellen, Karten und Abbildungen. Diese Arbeit hat in dieser oder einer ähnlichen Form noch nicht im Rahmen einer anderen Prüfung vorgelegen. Generative KI wurde ausschließlich zur stilistischen Überarbeitung des Textes eingesetzt.

Berlin, 31 Juli 2025



JONAS MORITZ MÜLLER, 0481804



ELIA RUBEN KRAEMER, 0489849



CARSTEN FRÉDÉRIC SCHLIEBEN, 0485860

1 Einführung

1.1 Motivation und Ziele

Alle Mitglieder unserer Gruppe verfügen über persönliche Erfahrungen mit digitalen Spielen, einschließlich strategischer Brettspiele wie Schach. Im Rahmen unseres Informatikstudiums sind wir zudem mit leistungsstarken Schach-Engines wie *Stockfish* sowie mit dem Paper über die Go-KI *AlphaZero* vertraut. Eine zentrale Motivation für dieses Projekt bestand darin, nachzuvollziehen, welche algorithmischen Prinzipien solchen Systemen zugrunde liegen und weshalb sie in der Lage sind, selbst die weltweit besten menschlichen Spielerinnen und Spieler deutlich zu übertreffen.

Unser Interesse galt dabei nicht nur den übergreifenden methodischen Ansätzen, sondern auch den spezifischen Techniken zur Effizienz- und Leistungssteigerung solcher KIs. Ein weiteres Ziel unserer Gruppe bestand darin, unsere im Studium erworbenen Kenntnisse praxisnah anzuwenden und im Rahmen des KI-Contests unter Beweis zu stellen. Dabei war es unser Anspruch, eine möglichst leistungsfähige KI zu entwickeln und im Wettbewerb bestmöglich abzuschneiden.

1.2 Aufbau des Berichts

Der vorliegende Bericht dokumentiert die wesentlichen Schritte und Überlegungen im Verlauf des Projekts. Er ist wie folgt gegliedert:

- Kapitel 2 beschreibt die Spielregeln und geht auf die Wahl der Programmiersprache sowie die Struktur unseres Projektmanagements ein
- Kapitel 3 präsentiert unsere zugrunde liegende Implementation, den selber entwickelten Client, sowie die KI-Implementierung und geht dabei detailliert auf die zentralen Optimierungen ein
- Kapitel 4 diskutiert offene Fragen, reflektiert über den Projektverlauf und gibt einen Ausblick auf mögliche Weiterentwicklungen.

2 Grundlagen

2.1 Spielregeln

Die im Folgenden dargestellten Spielregeln basieren vollständig auf dem Regelwerk für Turm und Wächter von Christoph Endres und Robert Wirth [Chr].

Das Spiel Turm und Wächter ist ein strategisches Zwei-Personen-Spiel. Es wird auf einem quadratischen Spielfeld mit 7×7 Feldern gespielt.

Jeder Spieler verfügt über einen identisch aufgebauten, farblich unterscheidbaren Spielsteinsatz, bestehend aus einem Wächterstein und sieben stapelbaren Turmsteinen. Die beiden Wächtersteine starten jeweils auf dem Mittelfeld der eigenen Grundlinie, welches entsprechend markiert ist. Ziel des Spiels ist es, entweder den gegnerischen Wächterstein zu schlagen oder das zentrale Startfeld des Gegners mit dem eigenen Wächterstein zu betreten.

Jeder Spieler führt abwechselnd einen Zug aus. In jedem Zug muss eine eigene Spielfigur bewegt werden – das kann entweder der Wächterstein, ein einzelner Turmstein oder ein Stapel eigener Turmsteine sein. Alle Figuren dürfen sich ausschließlich orthogonal, also horizontal oder vertikal, über das Spielfeld bewegen. Die Reichweite eines Spielzugs entspricht exakt der Höhe der bewegten Figur: Ein einzelner Turmstein oder der Wächter bewegt sich ein Feld weit, ein Turmstapel aus mehreren Steinen entsprechend seiner Höhe. Türme dürfen nicht übersprungen werden.

Ein wesentliches Spielelement ist das Stapeln und Entstapeln von Turmsteinen. Gelingt ein bewegter eigener Turmstein oder -stapel auf ein Feld mit einem weiteren eigenen Turmstapel, werden die Steine dort zu einem höheren Stapel zusammengeführt. Umgekehrt ist es auch möglich, einen Teil eines bestehenden Stapels abzuziehen und als neue Einheit zu bewegen. Ein einzelner Spielzug kann sowohl ein Entstapeln als auch direkt anschließendes Stapeln beinhalten.

Das Schlagen gegnerischer Figuren ist nur unter bestimmten Voraussetzungen erlaubt: Ein Wächterstein kann jede beliebige gegnerische Figur schlagen. Der Wächter selbst kann von jedem gegnerischen Turm geschlagen werden. Außerdem kann ein Turmstapel einen gegnerischen Turmstapel schlagen, sofern seine Höhe mindestens gleich groß ist.

Das Spiel endet, sobald ein Wächterstein geschlagen wurde oder sobald ein Wächter das Startfeld des gegnerischen Wächtersteins betritt.

2.2 Verwendete Programmiersprache

Wir entschieden uns frühzeitig für die Programmiersprache C++ – eine Wahl, die durch mehrere Faktoren motiviert war. Zum einen verfügten zwei Gruppenmitglieder über Vorkenntnisse aus einem vorausgegangenen Semester (Computer Graphics[Ale25]), in dem sie sich intensiv mit der Sprache und ihren Stärken auseinandergesetzt hatten. Zum anderen orientierten wir uns an etablierten KI-Engines wie Stockfish[Soub], die ebenfalls in C++ implementiert sind.

Ein zentrales Ziel unseres Projekts bestand nicht nur in der Umsetzung algorithmischer Konzepte, sondern auch im vertieften Verständnis hardwarenaher Optimierungstechniken. C++ erwies sich in diesem Zusammenhang als besonders geeignete Programmiersprache, da sie eine feingranulare Kontrolle über Ressourcen sowie über performancekritische Aspekte wie Speicherverwaltung und Datenstrukturen ermöglicht.

Von einer Implementierung in C sahen wir bewusst ab, da diese mit einem signifikant höheren Entwicklungsaufwand verbunden gewesen wäre – insbesondere aufgrund der im Vergleich zu C++ eingeschränkten Standardbibliotheken und des Fehlens moderner Sprachkonstrukte. Auch andere Hochsprachen wie Python oder Java wurden nicht weiter in Betracht gezogen. Zwar hätten sie eine vereinfachte Umsetzung komplexer Logikstrukturen – insbesondere im Bereich des maschinellen Lernens – ermöglicht, doch wären sie in Bezug auf Laufzeiteffizienz und hardwarenahe Kontrolle deutlich im Nachteil gewesen.

Da die Performanz unserer KI – insbesondere im Hinblick auf die Anforderungen des Wettbewerbs – eine zentrale Rolle spielte, erschien C++ als die am besten geeignete Wahl.

2.3 Projektmanagement

Die Kommunikation innerhalb unserer Gruppe erfolgte überwiegend über eine gemeinsame WhatsApp-Gruppe, in der wir uns regelmäßig über Fortschritte, Herausforderungen und Lösungsansätze austauschten. Bei erhöhtem Arbeitsaufkommen oder näher rückenden Abgabeterminen trafen wir uns zusätzlich regelmäßig zum gemeinsamen Arbeiten vor Ort in der Universität.

Zur Organisation und Koordination der Aufgaben nutzten wir GitLab als zentrale Plattform. In diesem Rahmen erstellten wir eine übersichtliche README-Datei, die

alle relevanten Informationen zur Ausführung des Programms enthält. Darüber hinaus pflegten wir einen CHANGELOG, in dem die wichtigsten Meilensteine und Änderungen am Projekt dokumentiert sind, sowie eine Lizenzvereinbarung.

Der typische Arbeitsablauf für einen Meilenstein begann mit einer gemeinsamen Zieldefinition in Form eines Meetings. Anschließend erstellten wir die entsprechenden Issues auf GitLab, ordneten diese über Merge Requests den verantwortlichen Gruppenmitgliedern zu und verfolgten den Fortschritt transparent über das Repository.

3 Ergebnisse

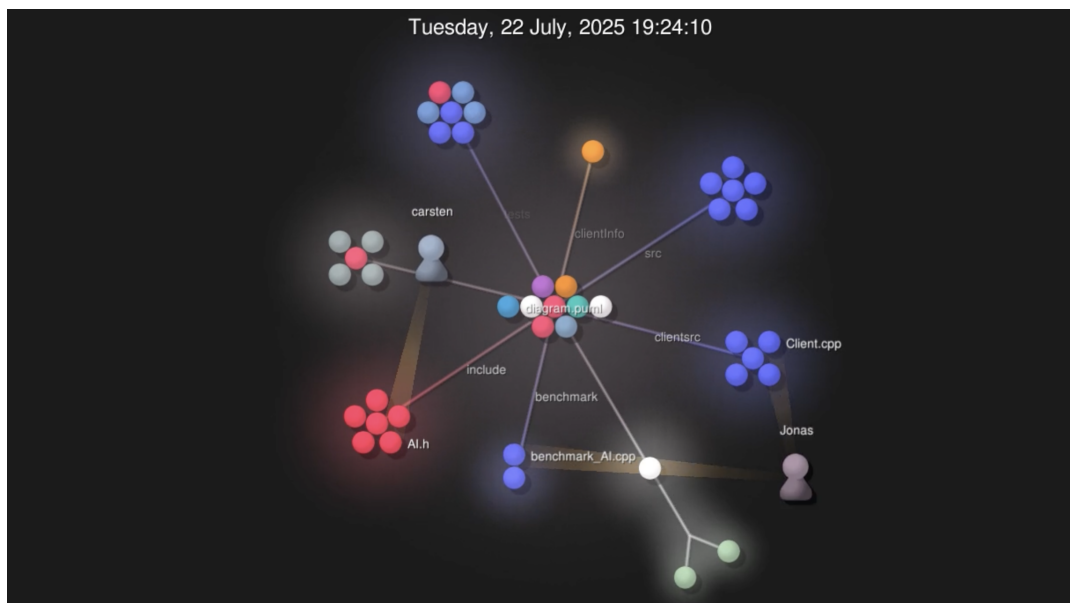


Abbildung 1: Klick auf das Bild öffnet das Video im GitLab Repository

Wurde mit dem Tool <https://gource.io/> und der Datei `gource_git.log` aus GitLab generiert

3.1 Dummy KI

Für die Implementierung unseres Zuggenerators entschieden wir uns frühzeitig für die Verwendung von Bitboards. Zum einen war dies durch die bereits erwähnte Bedeutung von Performance in unserem Projekt motiviert, zum anderen aus der Erfahrung heraus, dass grundlegende Strukturen, wie die Spiellogik, nur sehr schwer nachträglich zu ändern sind, insbesondere wenn bereits ein großer Teil der KI-Funktionalität darauf aufbaut.

3.1.1 Bitboards

Zur Vorbereitung recherchierten wir verschiedene Ansätze zur Implementierung von Bitboards. Hierbei orientierten wir uns primär an dem im ISIS verlinkten Journal [Bro14] sowie an den thematischen Ressourcen auf chessprogramming.org [Wik22]. Da sich die Spielfiguren in unserem Spiel jedoch deutlich einfacher und auf einem kleineren Spielfeld bewegen als im klassischen Schach, entschieden wir uns bewusst gegen komplexere Optimierungen wie den Einsatz von sogenannten Magic Bitboards [Wik24a]. Basierend auf unseren Recherchen und Anforderungen entwickelten wir ein eigenes Bitboard-Layout. Wir verwenden dabei 64 Bits zur Speicherung eines einzelnen Bitboards. Das Layout sieht wie folgt aus:

0	x	0	0	0	0	0	0	0	x
9	x	0	0	0	0	0	0	0	x
18	x	0	0	0	0	0	0	0	x
27	x	0	0	0	0	0	0	0	x
36	x	0	0	0	0	0	0	0	x
45	x	0	0	0	0	0	0	0	x
54	x	0	0	0	0	0	0	0	x
63	x								

Abbildung 2: Bitboard-Layout mit Bitindex auf der linken Seite

Wie in Abbildung 2 dargestellt, entschieden wir uns für ein Format mit sogenannten Separating Bits, welche als x dargestellt werden. Diese wurden ursprünglich eingeführt, da wir zu diesem Zeitpunkt noch über die Verwendung von Masken [Bro14] im Zuggenerator diskutierten und ein flexibles Layout bevorzugten. Zur Darstellung des Spielzustands nutzen wir ein Array aus 10 Bitboards. Die ersten acht Einträge dienen der Speicherung der Positionen der unterschiedlichen Turmartentypen. Die letzten beiden Bitboards enthalten die Zuordnung zur Spielerfarbe.

3.1.2 Zuggenerator

Der Zuggenerator funktioniert, indem wir für jeden Turm in jede Richtung abhängig von der Turmhöhe iterativ prüfen, ob das entsprechende Feld frei oder besetzt ist, da wenn dieses besetzt ist der Turm nicht weiter als dieses Feld gehen kann. Wir hatten uns wie erwähnt auch Masken angeschaut, aber wir sahen den Implementierungsaufwand als zu hoch an.

3.1.3 Tests und Benchmarks

Im Verlauf der Projektarbeit trafen wir als Gruppe die Entscheidung, Funktionalitäten, die nicht in direktem Zusammenhang mit der KI stehen, mithilfe externer Bibliotheken umzusetzen. Diese Entscheidung begründete sich vor allem durch unsere begrenzten personellen Ressourcen: Da wir nur ein kleines Team waren, wollten wir unsere verfügbare Zeit gezielt auf die Entwicklung und Optimierung der KI-Techniken konzentrieren.

Für das Testen der Korrektheit und Performance unserer Implementierung nutzten wir die externe C++-Bibliothek Catch2 [Org]. Diese ermöglichte es uns, strukturierte und übersichtliche Unit-Tests zu schreiben sowie aussagekräftige Benchmarks durchzuführen, was ohne diese deutlich aufwändiger gewesen wäre. Zum Einlesen von Testdaten im CSV-Format setzten wir einen externen CSV-Parser[La] ein. Zu Projektbeginn hatten wir mit fehlerhaften Dateieinträgen anderer Gruppen zu kämpfen, was uns unnötig Zeit beim Debugging kostete. Der Einsatz eines etablierten Parsers ermöglichte uns, den Fokus auf die inhaltliche Korrektur der Daten zu legen, anstatt uns mit potenziellen Parsing-Fehlern auseinanderzusetzen. In diesem Zusammenhang erstellten wir außerdem eine bereinigte Version der CSV-Testdateien, die wir allen anderen Gruppen zur Verfügung stellten [Mue25].

3.2 Basis KI

In diesem Abschnitt des Berichts konzentrieren wir uns auf die Aspekte, welche die Arbeit der KI nicht direkt, sondern indirekt beeinflussen. Dazu zählen insbesondere die Entwicklung des Clients, die Integration von Multithreading sowie umfangreiches Refactoring der Code-Basis.

3.2.1 ein eigener Client

Nachdem der Client und Server von Sarah zur Verfügung gestellt wurden, begannen wir mit der Implementierung der Kommunikation. Ursprünglich hatten wir geplant, die Ergebnisse unserer KI über eine einfache Pipe-Verbindung an den bestehenden Python-Client weiterzugeben. Nach einer Analyse des minimalistisch gehaltenen Python-Clients entschieden wir uns jedoch, einen eigenen C++-Client zu entwickeln. Dies brachte nicht nur Performance-Vorteile, sondern erlaubte uns auch, den Client flexibel zu erweitern.

Dabei stießen wir auf einige plattformabhängige Herausforderungen, insbesondere im Hinblick auf die Bereitstellung einer lauffähigen Windows-Binary für den Wettbewerb. Im Gegensatz zu Unix-basierten Systemen nutzt Windows seine eigene Netzwerk-API, was eine plattformübergreifende Kompilierung erschwerte. Mithilfe von Makros zur Betriebssystemerkennung und einer externen JSON-Bibliothek[Loh] gelang es uns dennoch, einen plattformkompatiblen Client zu entwickeln, der sich funktional an Sarahs Vorlage orientierte. Zur Absicherung gegen endlose Zugwiederholungen ergänzten wir den Client um eine einfache Drei-Zug-Schleifenerkennung.

Unsere Anwendung wird mit insgesamt 3 vorcompilierten Client-Binaries ausgeliefert – jeweils optimiert für Intel-, AMD- und generische x86-Architekturen. Die Hauptbinary unserer Anwendung ist der Client-Launcher, welcher zur Laufzeit die Prozessorarchitektur des Systems erkennt und anschließend automatisch die passende, optimierte Client-Binary (für Intel, AMD oder generisch) startet. Um diese Funktionalität zuverlässig sicherzustellen, wird dem Client-Launcher ein entsprechendes Manifest beim Kompilieren beigelegt. Dieses stellt sicher, dass beim Start der Anwendung die erforderlichen Berechtigungen angefordert werden, um zur Laufzeit Child-Prozesse erzeugen und ausführen zu dürfen.

3.2.2 Performance-Optimierung durch Parallelisierung

Im Rahmen weiterer Optimierungen untersuchten wir verschiedene Möglichkeiten, die Performance der Alpha-Beta-Suche zu verbessern. Multithreading stellte sich hierbei als naheliegender erster Schritt heraus. Darüber hinaus diskutierten wir auch die potenzielle Einbindung von GPU-Ressourcen zur parallelen Berechnung von Zugmöglichkeiten auf Bitboards. Erste Tests mit AdaptiveCPP[Hei] zeigten zwar, dass dies grundsätzlich möglich war, erwiesen sich jedoch als zu aufwendig hinsichtlich Setup, Plattformkompatibilität und Build-Tooling. Nach einem initialen Prototypen wurde dieser Ansatz daher

nicht weiterverfolgt.

Im Zuge unserer Recherchen zu Bitoperationen von C++20 (z.B. `std::count_zero()` im Vergleich zu `__builtin_ctz()` in GCC) stießen wir auf die neuen Standardbibliotheken `<execution>` und `<algorithm>`, welche ab C++20 eine parallele ausführbare Schleife zur Verfügung stellten. Um diese zu nutzen, mussten wir aber noch eine passende Multithreading-Bibliothek verlinken.

Nach dem Vergleich mehrerer Optionen – insbesondere Intel oneAPI Threading Building Blocks (TBB)[Int] und HPX[Soua] – entschieden wir uns für TBB, da die Integration im Kontext unseres Projekts deutlich einfacher war. Die Implementierung selbst gestaltete sich entsprechend unkompliziert: Die Bibliothek wurde über CMake eingebunden, und die parallele Ausführung der `traverseAlphaBeta`-Funktion wurde an der Wurzel des Suchbaums angestoßen. Dabei wurden für jeden Subbaum von der Root aus separate Threads erzeugt – in dem Umfang, den die verwendete Hardware zuließ.

Nach dem zweiten Meilenstein wurde uns der Einsatz von Multithreading innerhalb der KI untersagt. Das in diesem Zusammenhang erworbene Wissen konnten wir jedoch gezielt in die Optimierung und das Tuning unserer Evaluationsfunktion einfließen lassen

3.4.3.

3.2.3 Refactoring

Für die möglichen Züge hatten wir ein separates Array aus 56 Bitboards benutzt. Die Struktur dieses Arrays basiert auf folgender Überlegung: Für jede der acht möglichen Spielsteine speichern wir zunächst die Ausgangsposition sowie bis zu sechs potenzielle Zielpositionen – abhängig von der maximalen Zuglänge. Daraus ergibt sich:

$$8 \text{ Türme} \times (1 \text{ Ausgangsposition} + 6 \text{ mögliche Züge}) = 56 \text{ Bitboards.}$$

Während der Implementierung unserer KI fiel uns auf, dass die Alpha-Beta-Suche eine explizite Liste von Zügen benötigt, mit Zügen, welche direkt an Funktionen wie `makemove()` und `unmakemove()` übergeben werden müssen. Bisher hatten wir – wie zuvor beschrieben – die berechneten Züge in einem Array von Bitboards gespeichert, was für die effiziente Darstellung zwar geeignet war, sich jedoch als ungeeignet für die Suchlogik herausstellte.

Um alle möglichen Züge durchlaufen zu können, mussten wir diese Bitboards zunächst in eine explizite Liste konkreter Züge umwandeln – ein Schritt, der sowohl unnötige

Umrechnungen verursachte als auch die Codebasis unnötig komplex gestaltete.

Um dieses Problem zu lösen, entwickelten wir eine neue Datenstruktur: ein eigenes Move-Struct, das folgende Informationen enthält:

- die Startposition als Bitboard
- die Zielposition als Bitboard
- die Zuglänge

Infolgedessen ersetzten wir unser ursprüngliches Bitboard-Array durch ein predefined Array von Move-Objekten der Größe 32 (8 Türme $\times$ 1 Zuglänge $\times$ 4 mögliche Richtungen). Unsere Funktion `generateMoves()` speichert seither die generierten Züge direkt in dieses Array vom Game-Objekt.

Nach dem Aufruf von `generateMoves()` muss dieses Array lediglich an die Alpha-Beta-Suche übergeben werden – eine Umrechnung entfällt vollständig. Die Beschleunigung der Suche ist in Abbildung 3 zu sehen.

Im Rahmen dieser Umstellung wurden zudem mehrere Funktionen, die ausschließlich mit Zügen zu tun hatten, aus der Game-Klasse in die neue Move-Struktur ausgelagert, was die Modularität und Wartbarkeit unseres Codes deutlich verbesserte.

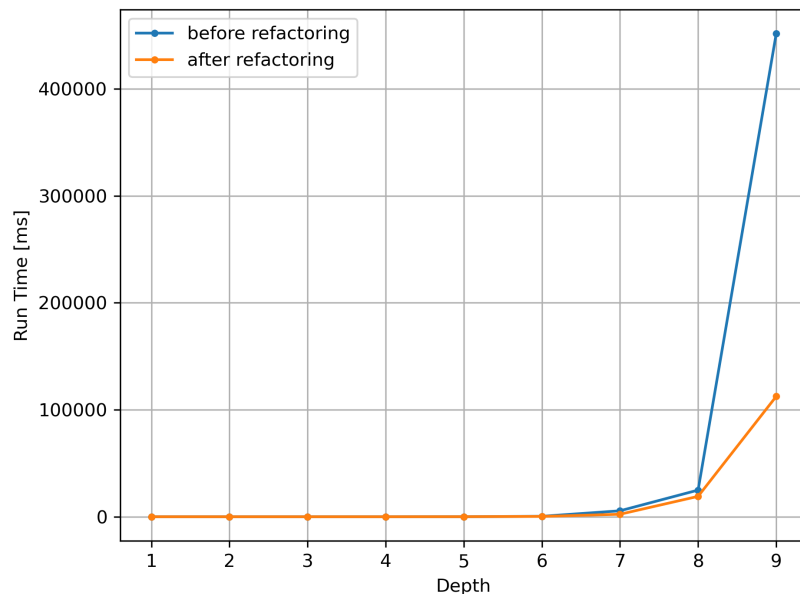


Abbildung 3: Beschleunigung durch refactoring

3.3 Erweiterte KI

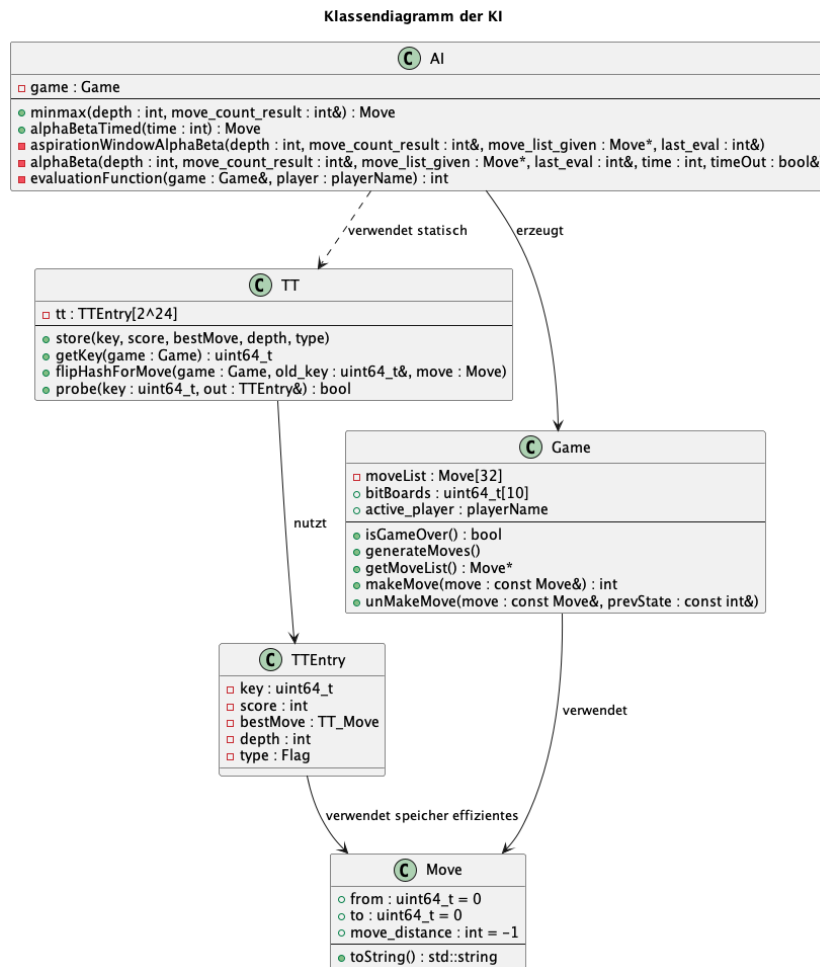


Abbildung 4: Klassendiagramm unsere aktuellen KI, wurde generiert mit <https://plantuml.com/de/> und `diagram.puml` vom Git Repository

Eine KI, die für spieltheoretische Szenarien entwickelt wird, basiert im Regelfall auf der Min-Max-Suche bzw. deren optimierter Variante, der Alpha-Beta-Suche. Beide Verfahren konnten wir ohne größere Schwierigkeiten implementieren; der Schwerpunkt unserer Arbeit lag jedoch auf der weiteren Optimierung dieser Suchalgorithmen – insbesondere mit dem Ziel, die Suchgeschwindigkeit zu erhöhen, um dadurch innerhalb der gegebenen Zeitlimits größere Suchtiefen zu erreichen. Zu diesem Zweck orientierten wir uns an klassischen Techniken und Algorithmen aus der Vorlesung sowie an bewährten Methoden aus der Entwicklung anderer Spiel-KIs.

3.3.1 Zugsortierung

Unsere Zugsortierung sortiert die Züge der ersten Suchtiefe basierend auf der Bewertung der vorherigen Suchen[Wik] mit geringerer Tiefe. Jeder Zug erhält dabei eine Bewertung und wird entsprechend in die Liste möglicher Züge eingeordnet. Dies führt zu einer in etwa 5–10 Prozent schnelleren Suche.

3.3.2 Aspiration Window

In unserer Implementierung von Aspiration Windows starten wir erst mit einem kleineren Suchfenster. Wenn dieses fehlschlägt, suchen wir entsprechend mit einem größeren Fenster [Mor]. Die Wahl, wie groß das erste Fenster und die darauffolgenden sind, bzw. wie häufig wir nicht mit vollem Fenster suchen, haben wir getroffen, indem wir mit verschiedenen Parametern Benchmarks laufen haben lassen und die besten ausgewählt haben. Die schnellste Version war bei uns, nur ein einziges relativ kleines Fenster zu haben und danach direkt die volle Suche zu machen, siehe Abbildung 10.

3.3.3 Transposition Tables - TT

Die Transposition Table stellte eines der größten und komplexesten Themenfelder bei der Entwicklung unserer KI dar. Sie begleitete uns vom dritten Meilenstein bis zur finalen Abgabe und erwies sich als sowohl konzeptionell als auch technisch besonders herausfordernd.

Bei der Implementierung orientierten wir uns sowohl an den in der Vorlesung vorgestellten Konzepten als auch am ergänzend verlinkten Dokument auf ISIS[BUH94]. Wir entschieden uns für eine globale, Header-only implementierte TT, die ihre Einträge zur Laufzeit speichern und beim nächsten Start wieder laden können sollte.

Die Größe der Tabelle beträgt 2^{24} Einträge. Für die Einträge wird eine eigene TTEntry-Struktur definiert, die folgende Informationen enthält:

- einen Hash-Key
- den besten gefundenen Score
- den zugehörigen Zug (TTMove)
- die Suchtiefe

- den Cutoff-Typ (Exact, Alpha, Beta)

Zur Speicherung der Züge implementierten wir ein kompaktes TTMove-Struct, das Start- und Zielpositionen in numerische Positionswerte konvertierte, um Speicherplatz zu sparen. Die Konvertierung zurück ins Bitboard-Format war bei Bedarf problemlos möglich. Die Hash-Werte basierten auf dem in der Vorlesung besprochenen Zobrist-Hashing, das es uns ermöglichte, den Hash-Wert effizient bei jedem `makeMove()` und `unMakeMove()` zu aktualisieren.

Die vollständige Transposition Table wurde am Ende jedes Programmlaufs als Binär-Stream in eine Datei geschrieben und zu Beginn eines neuen Laufs wieder eingelesen, um die Rechenleistung früherer Partien nutzbar zu machen.

Die größten Herausforderungen ergaben sich nicht aus der technischen Implementierung an sich, sondern aus der logischen Implementierung und dem Umgang mit Fehlverhalten, das auf die TT zurückzuführen war. Die KI zeigte in vielen Fällen gute Ergebnisse – unabhängig davon, ob die TT korrekt funktionierte oder nicht. Dabei muss erwähnt sein, dass falsche Implementierung in diesem Kontext eine falsche Grenzsetzung und/oder zu frühes Abbrechen der tieferen Suche zur Folge hatte. Deshalb führte die Nutzung der TT erst in ganz bestimmten Spielsituationen zu plötzlichen, schwer nachvollziehbaren Fehlentscheidungen, obwohl das Programm nicht abstürzte und nach außen hin korrekt weiterlief.

Besonders schwierig war es, diese Fehler gezielt zu testen und zu reproduzieren, da sich TT-bezogene Fehler selten deterministisch äußern. Ein zentrales Problem war etwa der Umgang mit TT-Hits und das korrekte Setzen der Flags, z.B. um zwischen „exact score“, „lower bound“ oder „upper bound“ zu unterscheiden.

Um die Fehler in unserer Transposition Table (TT) zu identifizieren, entschieden wir uns dazu, die Speicherung der TT sowohl zwischen einzelnen Spielen als auch zwischen Zügen zu entfernen. Zusätzlich erstellten wir einen neuen Entwicklungs-Branch, in dem wir das System von Grund auf neu aufbauten. Dabei übertrugen wir bereits entwickelte Features schrittweise und unterzogen sie jeweils einer intensiven Testphase. Durch dieses methodische Vorgehen konnten wir mehrere bislang unentdeckte Fehler aufdecken und beheben. Nach zahlreichen Iterationen und sorgfältigen Überlegungen waren wir schließlich überzeugt, eine logisch konsistente und funktionsfähige Version der Transposition Table implementiert zu haben. Etwa zu diesem Zeitpunkt stießen wir auf eine für uns besonders hilfreiche Quelle: eine öffentliche Schach-Engine[Qui], deren TT-

Implementierung exakt dem Ansatz entsprach, den wir nach unseren eigenen Überlegungen verfolgt hatten.

Rückblickend hätte es sich als sinnvoll erwiesen, die Transposition Table zunächst ohne Speicher-/Ladefunktionalität stabil zu implementieren, bevor weitere Komplexität hinzukam. Zudem stellte sich heraus, dass viele der Ressourcen zum Thema TT, abhängig von ihrer KI-Implementierung, unterschiedliche Ansätze verfolgten.

3.3.4 Principal Variation Search - PVS

Bei der Principal Variation[Wik24b] Search sucht man den ersten Zug mit einem Nullfenster ab. Dafür ist es wichtig, dass der beste Zug als erstes durchsucht wird. Dies wird gewährleistet durch die Transposition Table, in der wir unter anderem den besten Zug einer Stellung speichern. Sind wir auf einer schon bekannten Stellung stellen wir diesen bereits berechneten besten Zug an den Anfang unserer Move Liste.

3.4 Optimierte KI

Wichtige, wenn auch nicht unmittelbar zur eigentlichen Suchlogik zählende Aspekte unserer KI sind die Zusammenstellung der implementierten Komponenten sowie die Entwicklung einer effektiven Evaluationsfunktion. Beide Elemente – zusammen mit unserer Teilnahme am Wettbewerb – werden im Folgenden näher erläutert.

3.4.1 Auswahl der Komponenten

Nachdem wir alle oben genannten Features erfolgreich implementiert hatten, bestand der nächste Schritt darin, herauszufinden, welche Kombination dieser Komponenten die besten Ergebnisse liefert.

In Abbildung 5 erkennt man, wie viel Performance Gewinn die einzelnen Techniken im Vergleich zur Nutzung von nur Alpha-Beta bringen.

Da PVS nur in Verbindung mit den Transposition Tables funktioniert, ist in Abbildung 6 der Vergleich von PVS zu nur TT abgebildet.

In Abbildung 11 erkennt man eine Beschleunigung der Suche mit dem hinzufügen jeder KI Technik, weshalb auch jede Technik in der finalen KI implementiert ist.

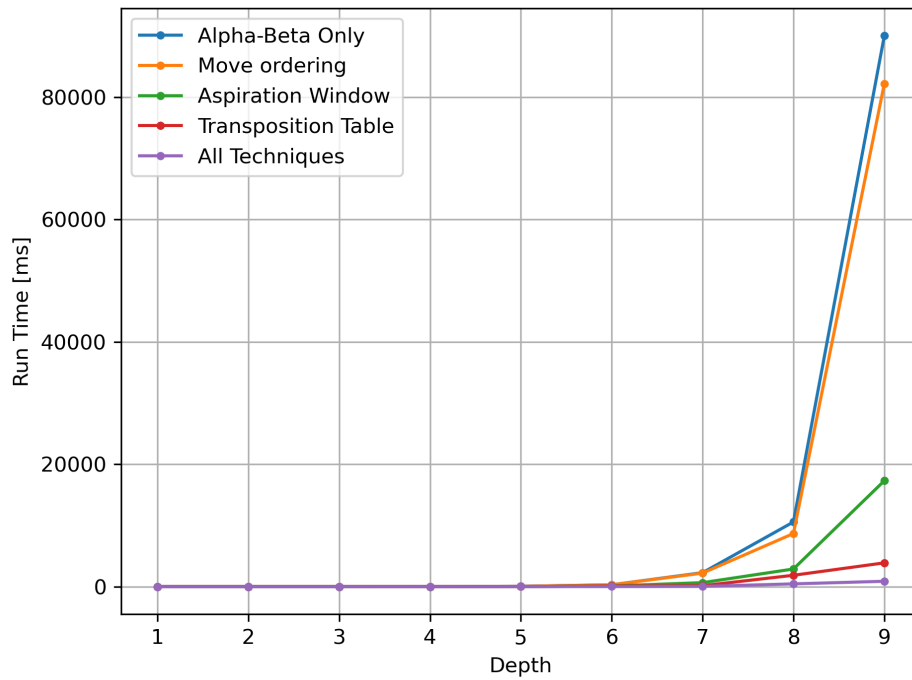


Abbildung 5: All Techniques compared to Only Alpha beta used

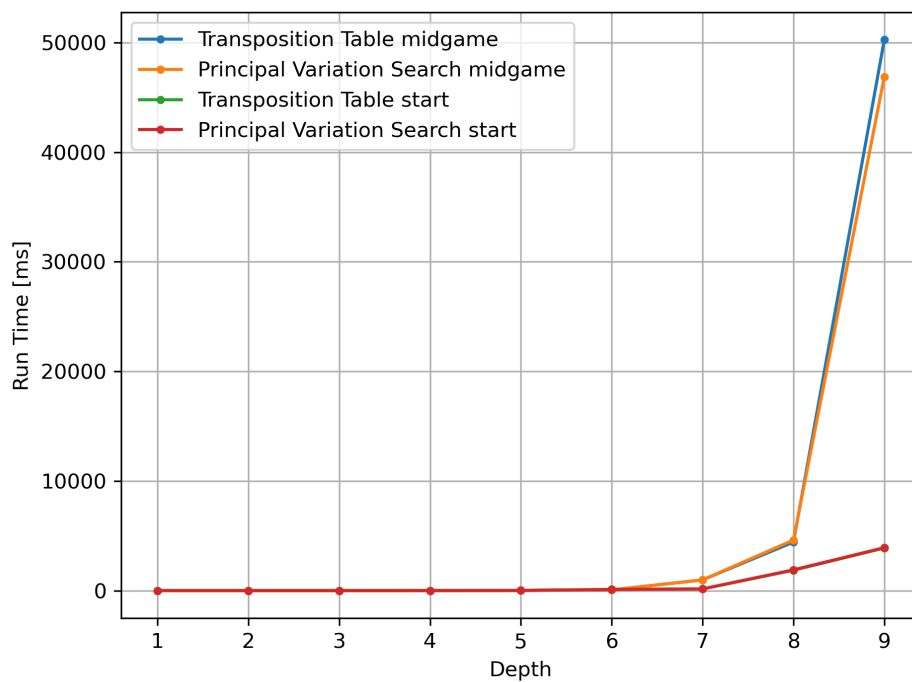


Abbildung 6: PVS

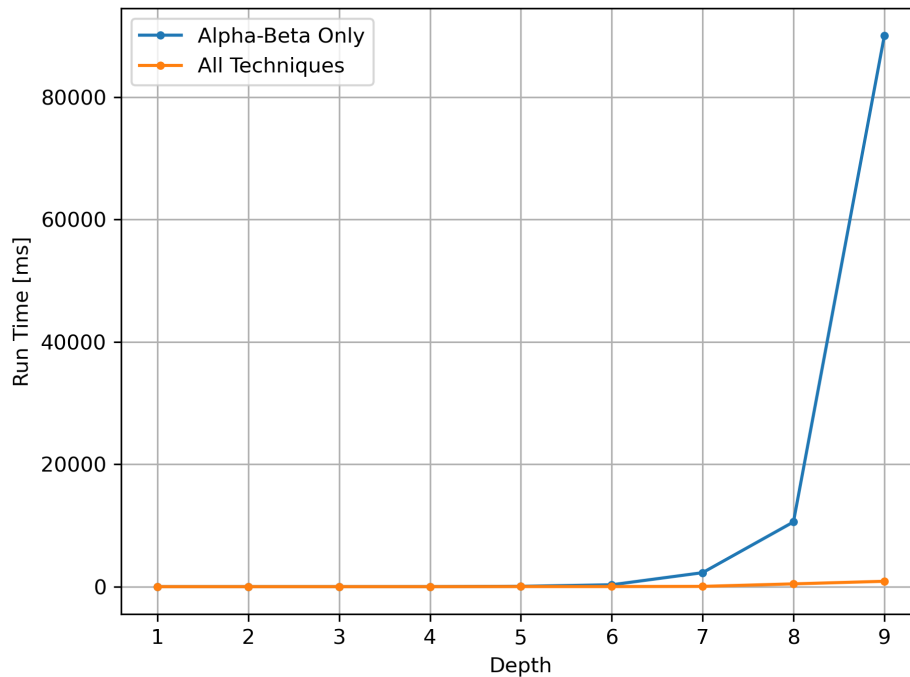


Abbildung 7: Alpha Beta Search vs All techniques used

Zum Abschluss ist der Unterschied zwischen reiner Alpha-Beta Suche und allen Techniken zusammen angewandt in Abbildung 7 zu sehen. Der performance Gewinn ist durch die Kombination aller Techniken am Ende doch beträchtlich. Vor allem wenn man betrachtet, wie sehr schon alleine Alpha-Beta die Suche beschleunigt hat 9.

Das Zeitmanagement haben wir dahingehend verbessert, dass der aktuelle Spielstand, sowie die Restzeit, zur Berechnung, wie viel Zeit wir unserer KI zur Suche geben, mit berücksichtigt wird.

3.4.2 Evaluation

Die Evaluationsfunktion unserer KI durchlief im Laufe der Entwicklung mehrere Iterationen, wobei sie sich von einer zunächst sehr einfachen Variante hin zu einer komplexeren und schließlich zu einer bewusst vereinfachten Version entwickelte.

Zu Beginn beschränkte sich die Bewertungslogik auf grundlegende Parameter wie die Anzahl der verbliebenen Spielfiguren, die Position des eigenen Wächters sowie die Entfernung zwischen dem nächstgelegenen gegnerischen Turm und dem eigenen Wächter.

In einer späteren Ausbaustufe orientierten wir uns an einer älteren Version der Stockfish-

Evaluationsfunktion[hxi20] – vor der Einführung von NNUE –, um unsere Funktion gezielt zu erweitern. Darauf basierend integrierten wir verschiedene neue Bewertungskriterien, etwa die Belohnung für das Schlagen gegnerischer Türme, die Bewertung taktisch sinnvoller Turmpaare, das Schützen eigener Figuren sowie das gezielte Angreifen oder Verteidigen des Wächterrands.

Im weiteren Verlauf reduzierten wir die Komplexität der Funktion jedoch wieder, um den Aufwand beim Finetuning zu minimieren und die Übersichtlichkeit der Gewichtung zu erhöhen. Dabei wurden vor allem jene Parameter beibehalten, die sich in praktischen Spielsituationen als besonders wirkungsvoll erwiesen hatten.

Unsere aktuelle Evaluationsfunktion setzt sich zusammen aus:

- Material
- Turmhöhen
- Position der Türme und des Guards auf dem Spielfeld
- Position der Türme zum eigenen und gegnerischen Guard

Alle genannten Komponenten wurden mit gewichteten Faktoren kombiniert. Die Gewichtung variierte abhängig vom Spielverlauf, wobei wir zwischen Midgame und Lategame unterschieden. Die Übergänge zwischen den Spielphasen wurden linear interpoliert – basierend auf der Anzahl verbleibender Figuren – sodass sich die Gewichtung dynamisch an die Spielsituation anpasste. Dieser Aspekt wurde aus der älteren Stockfish-Version übernommen und erwies sich als besonders vorteilhaft.

Diese verbesserte Evaluationsfunktion stellte einen bedeutenden Schritt in Richtung strategischer Tiefe dar und ermöglichte unserer KI fundiertere Entscheidungen im mittleren und späten Spielverlauf.

3.4.3 Tuning der Evaluationsfunktion

Unsere Bewertungsfunktion haben wir durch Tuning der Parameter verbessert. Wir haben ein großes Turnier vieler KIs mit verschiedenen Parametern ausgerichtet, wobei immer die KI mit den meisten Siegen ausgewählt wurde und die KIs in der nächsten Runde um diese Parameter herum ihre Werte hatten. So setzen sich mit der Zeit die besten Parameter durch. Wir hatten dabei die benötigte Rechenzeit unterschätzt und deshalb noch Multithreading und einen vorhergehenden Filter eingebaut. Der Filter sorgt dafür, dass

nur Parameter, die gegen unsere ursprüngliche KI gewinnen, zugelassen werden, damit sehr schlechte Parameter direkt aussortiert werden. Dennoch war die Laufzeit sehr lang, weshalb es hier sinnvoller gewesen wäre, zu recherchieren, was es für Algorithmen gibt, mit denen man Parameter trainieren kann, anstatt eigenständig etwas zu bauen. Dies wäre dann wahrscheinlich ein besseres Verfahren gewesen, womit man dann auch evtl. mehr Parameter hätte berücksichtigen können und somit eine komplexere Bewertungsfunktion hätte nutzen können. Unserer Meinung nach ist die Bewertungsfunktion leider aktuell der schwächste Teil unserer KI.

3.4.4 Teilnahme am Wettbewerb

Mit dieser KI-Implementierung nahmen wir erfolgreich am ersten Wettbewerb teil und konnten diesen gewinnen. In Abbildung 8 kann man die Ergebnisse der einzelnen Paarungen sehen. Es wurden immer zwei Spiele pro Paarung gespielt, sodass jedes Team einmal anfangen konnte. Die Anfangende KI ist rot. Man sieht also, dass wir von den 4 Begegnungen 3 komplett gewonnen und in einer gewonnen sowie verloren haben. Es zeigte damit, dass unsere Konzentration auf einfache, aber funktionierende Strategien und Performanceoptimierung Früchte trug und ein solides Fundament für spätere Weiterentwicklungen bot.

Runde 1	Grund	Runde 2	Grund	Runde 3	Grund	Runde 4	Grund
AB-AC	Sieg	AD-K	TimeOut	AC-AD	Sieg	AD-J	No Move
AC-AB	Sieg	K-AD	TimeOut	AD-AC	Sieg	J-AD	No Move
AD-AF	TimeOut	G-AC	TimeOut	J-K	TimeOut	AC-AF	Sieg
AF-AD	Sieg	AC-G	TimeOut	K-J	TimeOut	AF-AC	TimeOut
G-J	TimeOut	AB-J	TimeOut	G-AF	Sieg	AB-K	Sieg
J-G	TimeOut	J-AB	TimeOut	AF-G	Sieg	K-AB	Sieg
K-P	Invalid	P-AF	Sieg	AB-P	Invalid	G-P	Invalid
P-K	Sieg	AF-P	Sieg	P-AB	Invalid	P-G	TimeOut

Abbildung 8: Einzelne Spielergebnisse des 1. Contest.

4 Diskussion und Ausblick

Im Verlauf des Projekts ist es uns gelungen, ein stabiles, schnelles und effizientes Grundgerüst einer KI zu entwickeln. Die Architektur ist so gestaltet, dass sie leicht erweiterbar ist – insbesondere im Hinblick auf die Evaluationsfunktion, die aktuell noch relativ einfach gehalten ist. Durch gezielte Erweiterungen und feinere Heuristiken besteht hier großes Potenzial, die Spielstärke unserer KI deutlich zu verbessern.

Eine der größten Fehler im Projektverlauf bestand darin, KI-bezogene Änderungen ohne ausreichende Tests direkt in den main-Branch zu integrieren. Gerade bei KI-Systemen ist es oft schwierig, auftretende Fehler rückwirkend genau zu lokalisieren, da fehlerhafte Entscheidungen nur schwer auf einen Programmierfehler zurückzuführen sind, sondern häufig aus komplexem Zusammenspiel mehrerer Faktoren resultieren.

Als Reaktion auf dieses Problem begannen wir gegen Ende der Entwicklungsphase, jede auffällige Fehlentscheidung, die uns beim Spielen auffiel, als Testfall zu erfassen. Daraus entstand ein wachsender Katalog an konkreten Spielsituationen, der uns half, die Robustheit und Zuverlässigkeit der KI sukzessive zu verbessern. Diese Herangehensweise hat sich als sehr effektiv erwiesen, da sie nicht nur zur Fehlervermeidung beitrug, sondern auch die langfristige Wartbarkeit des Codes erhöhte.

Unsere KI ist stabil, schnell und erreicht in der aktuellen Form eine solide Spielstärke. Dennoch sehen wir mehrere Potenziale für Weiterentwicklungen und Optimierungen, die im Rahmen dieses Projekts nicht mehr vollständig umgesetzt werden konnten.

Einige der Funktionen und Techniken, die wir gerne noch getestet hätten, sind unter anderem:

Die Implementierung von Bayes Trees zur probabilistischen Bewertung von Zugfolgen, sowie die Quiescence Search, um sogenannte „horizons effect“-Probleme zu minimieren.

Das vermutlich größte verbleibende Optimierungspotenzial liegt jedoch in der Evaluationsfunktion selbst. Während unsere aktuelle Bewertung auf klassischen heuristischen Faktoren basiert, wäre eine vielversprechende Weiterentwicklung die Integration eines neuronalen Netzwerks, wie es beispielsweise in modernen Stockfish-Versionen[[Soub](#)] durch NNUE (Efficiently Updatable Neural Network Evaluation) zum Einsatz kommt.

Ein solches System könnte deutlich differenzierter bewerten und in Kombination mit unserem performanten Suchgerüst die Spielstärke der KI auf ein neues Niveau heben.

5 Literatur

- [Ale25] Marc Alexa. *Computer Graphics 1 (WS 25/26)*. Technische Universität Berlin, Fachgebiet Computergrafik. 2025. URL: <https://www.cg.tu-berlin.de/teaching/cg1-2526> (besucht am 31.07.2025).
- [Bro14] Cameron Browne. „Bitboard Methods for Games“. In: *ICGA Journal* 37.2 (2014), S. 67–84. URL: https://www.researchgate.net/publication/279031895_Bitboard_Methods_for_Games (besucht am 31.07.2025).
- [BUH94] Dennis Breuker, Jos W. H. M. Uiterwijk und H. Jaap van den Herik. „Replacement Schemes for Transposition Tables“. In: *ICGA Journal* 17.4 (1994). DOI: 10.3233/ICG-1994-17402.
- [Chr] Christoph Endres and Robert Wirth. *Turm und Wächter: Ein Strategiespiel für zwei Spieler*. URL: <http://www.turm-und-waechter.de/tuw-de.pdf> (besucht am 31.07.2025).
- [Hei] Open Source primarily led by Heidelberg University. *AdaptiveCpp*. URL: <https://github.com/AdaptiveCpp/AdaptiveCpp> (besucht am 31.07.2025).
- [hxi20] hxim. *Stockfish Evaluation Guide*. 2020. URL: <https://hxim.github.io/Stockfish-Evaluation-Guide/> (besucht am 31.07.2025).
- [Int] Intel. *oneAPI Threading Building Blocks*. URL: <https://github.com/uxlfoundation/oneTBB> (besucht am 31.07.2025).
- [La] Vincent La. *Vince’s CSV Parser*. URL: <https://github.com/vincentlaucsb/csv-parser> (besucht am 31.07.2025).
- [Loh] Niels Lohmann. *nlohmann/json*. URL: <https://github.com/nlohmann/json> (besucht am 31.07.2025).
- [Mor] Bruce Moreland. *Aspiration Windows*. URL: <https://web.archive.org/%20web/20070705134903/www.seanet.com/~brucemo/topics/aspiration.htm> (besucht am 31.07.2025).
- [Mue25] Jonas Moritz Mueller. *Beitrag im ISIS-Forum der TU Berlin im Kurs [SoSe 2025] PJ KI: Zuggenerator Tests überarbeitet*. Interner Forenbeitrag auf der Lernplattform ISIS der TU Berlin, Zugriff nur für angemeldete Nutzer. 2025. URL: <https://isis.tu-berlin.de/mod/forum/discuss.php?d=654817#p1161152> (besucht am 31.07.2025).

- [Org] Catch Org. *Catch2*. URL: <https://github.com/catchorg/Catch2> (besucht am 31.07.2025).
- [Qui] Quintium. *QuintBot*. URL: <https://github.com/Quintium/QuintBot/tree/master> (besucht am 31.07.2025).
- [Soua] Open Source. *HPX*. URL: <https://github.com/STELLAR-GROUP/hpx> (besucht am 31.07.2025).
- [Soub] Open Source. *Stockfish – A free and strong UCI chess engine*. Accessed on GitHub. Licensed under GNU GPL-3.0. URL: <https://github.com/official-stockfish/Stockfish> (besucht am 31.07.2025).
- [Wik] Wikipedia contributors. *Iterative Tiefensuche – Wikipedia*. URL: https://de.wikipedia.org/wiki/Iterative_Tiefensuche (besucht am 31.07.2025).
- [Wik22] Chess Programming Wiki. *Bitboards*. 2022. URL: <https://www.chessprogramming.org/Bitboards> (besucht am 31.07.2025).
- [Wik24a] Chess Programming Wiki. *Magic Bitboards*. 2024. URL: https://www.chessprogramming.org/Magic_Bitboards (besucht am 31.07.2025).
- [Wik24b] Chess Programming Wiki. *Principal Variation Search*. 2024. URL: https://www.chessprogramming.org/Principal_Variation_Search (besucht am 31.07.2025).

Anhang

Rohdaten der benchmarks

start_minmax = [0.00720575, 0.168874, 3.84429, 93.9849, 2116.16, 48654]

before_refactoring = [0.0152113, 0.069251, 0.532713, 3.83, 47.242, 411.096, 5546.97, 24905.2, 451579.2]

after_refactoring = [0.007013, 0.054783, 0.653947, 4.75891, 55.7378, 303.771, 2320.14, 19038.4, 112486.8]

start_only_ab = [0.00647345, 0.0494452, 0.626164, 4.40996, 60.1247, 309.373, 2287.28, 10575.8, 90031.8]

mg_only_ab = [0.00371836, 0.0256057, 0.336236, 0.979121, 21.9053, 121.779, 1764.81, 7929.22, 121642.2]

start_move_ordering = [0.00656048, 0.0508619, 0.608776, 4.38541, 58.7841, 294.176, 2214, 8683.16, 82208.4]

mg_move_ordering = [0.00374533, 0.0149503, 0.344446, 0.809487, 20.9717, 111.11, 1707.96, 7751.84, 123765.6]

start_aspiration = [0.0113933, 0.0519263, 0.331843, 2.35454, 19.7191, 114.497, 654.565, 2889.16, 17308.1]

mg_aspiration = [0.0382114, 0.0163717, 0.172598, 0.509722, 6.73752, 39.8257, 236.253, 1803.29, 12756.5]

start_asp_parameters = [915.021, 873.977, 902.127, 878.238, 884.559, 922.601, 1002.1, 1212.64, 1038.23]

asp_labels = ["1-1-X", "1-20-X", "2-20-4", "3-20-4", "3-20-2", "1-50-X", "1-200-X", "1-400-X", "1-800-X"]

start_tt = [0.00936433, 0.0524433, 0.597691, 4.03362, 22.5398, 110.008, 156.861, 1860.74, 3889]

mg_tt = [0.00389019, 0.0163408, 0.337755, 0.798829, 20.8631, 66.0618, 982.228, 4428.96, 50247.3]

start_tt_pvs = [0.00685683, 0.0507997, 0.770713, 4.66928, 23.1839, 108.857, 158.062, 1896.8, 3920.07]

mg_tt_pvs = [0.003644, 0.0164863, 0.370185, 0.807852, 21.4738, 70.0198, 980.408, 4618.14, 46883.8]

start_all = [0.00662755, 0.0484165, 0.33337, 2.36261, 17.1823, 17.9047, 44.7605, 454.413, 878.238]

mg_all = [0.00365034, 0.0151792, 0.178207, 0.652046, 9.68554, 42.3462, 154.364, 706.084, 2873.95]

Kombination der Techniken (Tiefe 4 - 8)

alpha_beta = [5.56, 73.45, 565.68, 7409, 25636]

refactoring = [2.31, 27.16, 147.16, 1140, 9456]

tt = [1.72, 26.09, 44.56, 60.84, 622.42]

asp_win = [1.14, 8.05, 9, 22.63, 235.01]

pvs = [1.2, 8.03, 8.77, 22.61, 231.5]

weitere Plots

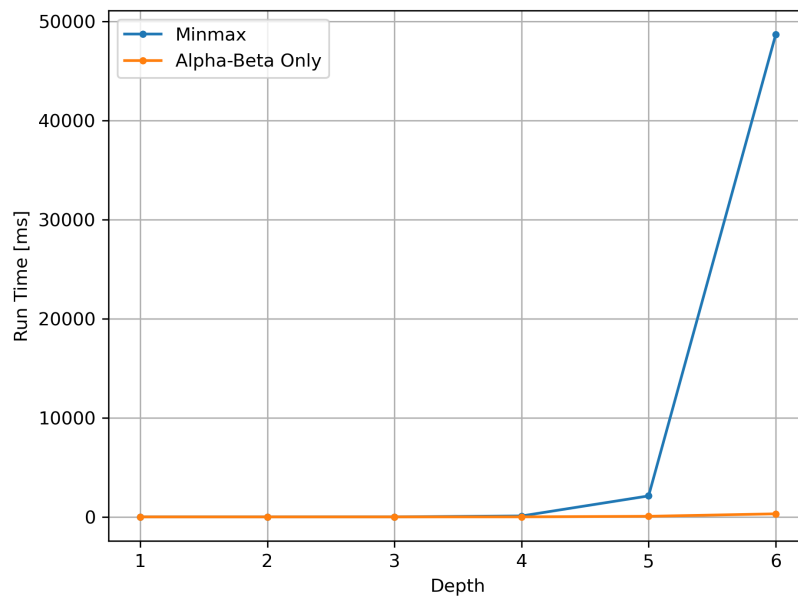
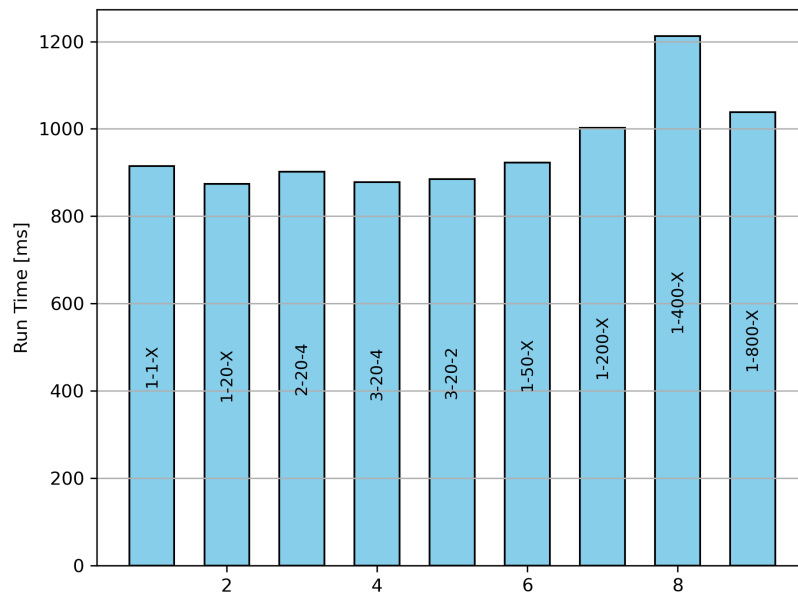


Abbildung 9: Minmax vs Alpha-Beta

Abbildung 10: verschiedene Paramter der Aspiration Windows, Tiefe 9
maximale retries - Vergrößerungsfaktor - aspiration margin

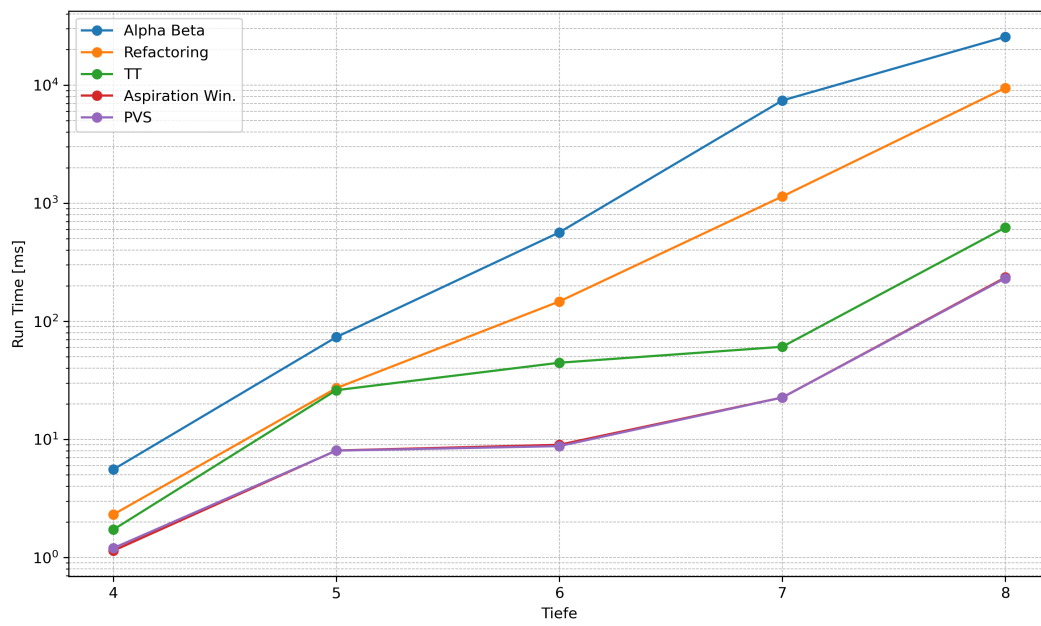


Abbildung 11: Techniken kombiniert angewandt